



Sponsoring Societies:



Regular Article

High-Performance Telemetry Visualization: Native C++ Architectures vs. Standard Web-Stack Baselines in Factory Workstations

Submission ID 255cd59d-9a38-4e92-a822-90dff5748b8f

Submission Version Initial Submission

PDF Generation 16 Jul 2026 12:06:28 EST by Atypon ReX

Authors

Mr. Divyam Agarwal
Corresponding Author
Submitting Author

 [ORCID](https://orcid.org/0009-0000-2992-5873)
<https://orcid.org/0009-0000-2992-5873>

Affiliations
• Delhi Technological University

Additional Information

Keywords
Area 2: Communications and Networking for IoT / Machine-to-Machine Communications
Machine-to-Machine Communications

Area 4: Services, Applications, and Other Topics for IoT / Cyber-Physical Systems
Cyber-Physical Systems

Area 4: Services, Applications, and Other Topics for IoT / Real-Time Systems
Real-Time Systems

Area 6: Distributed Computing and Autonomous IoT Systems / Edge Processing Systems
Edge Processing Systems

Files for peer review

All files submitted by the author for peer review are listed below. Files that could not be converted to PDF are indicated; reviewers are able to access them online.

Name	Type of File	Size	Page
A_Sample_Article_Using_IEEEtrancls_for_IEEE_Journals_and_Transactions (2).pdf	Main Document - PDF	147.5 KB	Page 3

High-Performance Telemetry Visualization: Native C++ Architectures vs. Standard Web-Stack Baselines in Factory Workstations

Divyam Agarwal

51

Abstract—In modern Industry 4.0 environments, factory monitoring workstations (“thick clients”) are tasked with visualizing high-frequency, asynchronous telemetry data in real-time. Despite having adequate computing and graphics resources, developers frequently default to deploying Web-based dashboards wrapped in browser engines. While a naive DOM/WebSocket implementation introduces severe frame-rate instability, we demonstrate a fascinating architectural trade-off when compared against a Full-Stack Optimized web client (Node.js + Canvas). When comparing the web-based frameworks, the Full-Stack Web (Node.js) architecture proved highly efficient, requiring only 83 MB for its routing layer. In stark contrast, the Python Flask backend suffered significant memory bloat, reaching 175 MB within 60 seconds. Furthermore, under extended runtime, we additionally observed catastrophic memory ballooning. Because of an invisible concurrency deadlock between Python’s cooperative eventlet async workers and unpatched blocking C-extensions (like Paho MQTT’s native OS threads), the Python dispatcher starved under load. Consequently, the outbound message queues grew infinitely, rapidly leaking past 1 GB of RAM during an informal, single 5-minute extended run. This highlights a fundamental weakness in Python’s fragmented async ecosystem for high-frequency telemetry routing, as a single missing monkey-patch call causes catastrophic failure. However, the web architecture cannot avoid the fixed cost of hosting a browser engine, incurring a massive 498 MB total memory footprint compared to the Native C++ application’s 199 MB. This paper argues that the choice between Native and Web architectures in Industry 4.0 is a strict trade-off: web architectures decouple routing for rapid development, but unified native architectures are highly advantageous for deployments constrained by absolute RAM limits. Whether this memory advantage is decisive in practice will depend entirely on the RAM headroom required by concurrent edge-AI workloads.

Index Terms—Edge Computing, Factory Automation, GTK4, Industrial Internet of Things, MQTT, Node.js, Python, Telemetry, WebSockets.

I. INTRODUCTION

THE monitoring of real-time factory floor data requires interfaces that are highly responsive and capable of absorbing massive data floods without lagging or crashing. As Industry 4.0 relies increasingly on edge computing for real-time industrial automation [1], telemetry systems have had to evolve to handle sustained, high-frequency continuous data streams.

A common trend in industry is to build these interfaces using rapid-development web frameworks (like Python Flask) and display them locally on a workstation via a web browser. While easy to develop, a naive DOM-based implementation of this “web-stack on a workstation” introduces severe latency.

As our results will show, while the backend CPU routing bottleneck of frameworks like Flask can be mitigated by switching to faster asynchronous runtimes (e.g., Node.js), the frontend clients still incur massive memory overhead due to the fixed architectural cost of hosting the browser engine.

Furthermore, modern factory monitoring workstations are frequently required to run heavy concurrent workloads, such as real-time object detection models (e.g., YOLOv8 [2] when run in CPU-only edge deployments) for automated industrial quality inspection [3]. Because these AI quality-control workloads demand maximum computational resources, it is imperative that the telemetry visualization interface consumes as little overhead as possible. This paper proposes a return to highly optimized, Native C++ architectures (utilizing GTK4) to completely bypass the massive browser memory tax and reclaim critical RAM headroom.

A. Reproducibility and Open Data

To ensure transparency and reproducibility, all benchmark scripts, the MQTT publisher, the C++ application source, the Node.js implementation, and raw htop capture logs for the $N = 10$ trials are available at: <https://github.com/Inuth0603/flask-mqtt-vs-native> (Note: The Node.js implementation was added after the repository was initially named, and is located in the `node_backend` directory).

II. METHODOLOGY

To evaluate the architectural efficiency of telemetry visualization, we simulated a high-frequency data flood representing factory sensor output. We compared a Native C++ Interface against three Web Interfaces: a standard DOM implementation (Flask backend), a Canvas implementation (Flask backend), and a Full-Stack Optimized implementation (Node.js backend + Canvas).

A. Experimental Setup & Workload Parameters

To ensure strict statistical validity, we conducted $N = 10$ independent trials for each architecture. Each trial consisted of a 60-second sustained data flood. A single Python publisher asynchronously published packets to an Eclipse Mosquitto broker at a pinned rate of **5,000 MQTT messages per second** (QoS 0, variable-length plain-text payloads of ~ 40 -50 bytes). The publisher, broker, and subscriber all ran concurrently on the same host, which means hardware contention was present, though we strictly measured metrics for the application processes themselves. System metrics (average CPU% and average Memory Resident Set Size) were sampled at 1-second

intervals using `htop` across all associated processes. Note that CPU% is reported on a per-core basis (where 100% = 1 fully saturated core, out of a maximum 1400%). Additionally, because QoS 0 is a fire-and-forget protocol, we did not strictly verify end-to-end delivery rates at the subscriber level; it is possible that silent message drops occurred at the TCP/broker layer under load, which could artificially lower CPU usage for subscribers that dropped more messages.

All benchmarks were executed on a standardized testbed: an Intel Core i7 processor (14 cores), 16GB DDR5 RAM, running Ubuntu 24.04 LTS via the Windows Subsystem for Linux (WSL2).

B. System Architecture & Data-Path Comparison

To understand the latency differences, we compare the critical data paths of both architectures in Table I.

TABLE I
CRITICAL DATA PATH COMPARISON

Layer / Phase	The Long Path (Web Baseline)	The Short Path (Native C++)
Ingress	MQTT TCP	MQTT TCP
Broker Intermediary	Mosquitto Broker	Mosquitto Broker
Data Router/IPC	Python Interpreter (Flask + WebSockets)	Direct Memory Copy (Mutex Shared State)
UI Engine / Renderer	WebBrowser Engine (DOM/CSS Tree)	GTK4 UI Engine (Direct-to-Hardware Draw)

While each transition step in the Web baseline introduces context switching, TCP overhead, and serialization delay, the native application completely bypasses these network layers. Because the native application operates entirely within a single, unified address space, the routing overhead is reduced to a simple memory write.

C. Thread-Safe UI Decoupling (The Native Approach)

A naive native GUI that attempts a screen redraw for every incoming packet will saturate the event loop, causing severe congestion and driving CPU utilization to nearly 100%. To resolve this, our optimized application decouples the network operations from the main rendering loop using a rate-limiting message coalescing strategy:

- 1) **Continuous Network Ingestion:** The background MQTT network thread continuously listens to the Mosquitto broker, swallowing thousands of high-frequency sensor packets per second without blocking.
- 2) **Thread-Safe Mutex Lock & Write:** The network thread briefly acquires a `std::mutex`, updates a shared state string, and immediately drops the lock to ensure negligible thread contention.
- 3) **Rate-Limited UI Polling:** Completely independent of the incoming data rate, a UI timer wakes up exactly 60 times a second to synchronize with standard monitor refresh cycles (16.6ms intervals).

- 4) **Single-Frame Redraw:** The main GTK thread acquires the mutex, copies the latest coalesced state, releases the lock, and triggers a single, unified frame redraw.

During implementation, we evaluated two distinct design patterns for thread communication. We chose **Frame-Rate Polling** because the UI main loop polling the shared state using a timer callback achieves automatic message coalescing. In contrast, Asynchronous Dispatching under severe telemetry floods can trigger dispatcher queue saturation, reverting the app to a high-CPU state unless manual rate-limiting is added.

D. Measurement Constraint (The WSL2 Handicap)

The benchmark was executed in a virtualized WSL2 environment lacking hardware graphics acceleration. Consequently, the Native C++ application was forced into CPU-bound software rendering via Cairo, artificially inflating its CPU metrics. However, Chromium's compositor is heavily GPU-dependent in ways Cairo software rendering is not, meaning the WSL2 environment likely penalizes the web client disproportionately more than the native one. We acknowledge this as a disclosed limitation of the test environment. Notably, this confound cuts unevenly across our results: the FPS and CPU findings are the ones most exposed to it, while the memory-tax finding is less affected by this asymmetry.

III. RESULTS

The performance divergence between the two paradigms becomes stark when looking at total resource allocation and responsiveness during peak telemetry floods.

A. Comparative Performance Metrics

Averages recorded across $N = 10$ trials (60s duration, 5,000 msg/sec load) are presented in Table II. All $\pm$ values denote Standard Deviation (σ).

B. Total System Memory Footprint

Under true extreme load, the Full-Stack Node.js backend RES grows to ~ 83 MB, reflecting V8's highly efficient asynchronous network buffering overhead. However, the true footprint of the Web Interface is completely obscured until you count the rendering client. Chromium-wrapped web clients spawn isolated, sandboxed processes for rendering and JavaScript execution. Regardless of how efficiently the dashboard DOM is constructed, the fixed baseline cost of hosting *any* browser engine creates a permanent memory tax exceeding 400 MB in our observations (totaling ~ 498 MB across the application).

In contrast, the C++ GTK4 application operates within a single memory space. While its memory footprint grew to 199 MB under the intense 5,000 msg/sec barrage, it still completely avoids the secondary browser tax, saving nearly 300 MB of physical memory compared to the Full-Stack Node.js web stack (and nearly 390 MB compared to the Flask web stack). This leaves critical RAM available for adjacent heavy edge computing processes.

TABLE II
COMPARATIVE PERFORMANCE METRICS

Metric	Native C++ (GTK4)	Naive Web (Flask/DOM)	Optimized Web (Flask/Canvas)	Full-Stack Web (Node.js/Canvas)
Routing Backend Memory (RES)	N/A (Unified)	175 MB $\pm$ 4 MB	175 MB $\pm$ 4 MB	83 MB $\pm$ 3 MB
Browser Renderer Memory (RES)	N/A (Unified)	412 MB $\pm$ 14 MB	411 MB $\pm$ 13 MB	415 MB $\pm$ 15 MB
Total Application Memory (Backend + Renderer)	199 MB $\pm$ 2 MB	587 MB $\pm$ 18 MB	586 MB $\pm$ 17 MB	498 MB $\pm$ 18 MB
Router/Backend CPU (native = whole app)	49.6% $\pm$ 1.5%	Backend Only: 53.9%	Backend Only: 53.9%	Backend Only: 24.7%
UI Frame Rate	60 FPS (Vsync)	<10 FPS (Lag)	60 FPS (Vsync)	60 FPS (Vsync)

*Note: The C++ Router/Backend CPU metric includes the entire application's software UI rendering, whereas the Web CPU metrics only capture the backend routing process, rendering them incomparable for holistic CPU efficiency. We report significance testing for the Native vs. Full-Stack Web total memory footprint comparison, confirmed via a Welch's t-test ($t(9.2) = 51.5, p < 0.001$).

C. Incomparable CPU Scopes (Backend vs. Full-Stack)

The 49.6% CPU metric for the Native C++ application encompasses the entire system workload: network ingestion, memory state synchronization, and heavily CPU-bound GTK4/Cairo software rendering. Conversely, the 24.7% metric for Node.js only captures the backend routing process. The Chromium renderer's CPU footprint was isolated in a separate process tree and omitted from the backend metric. Therefore, while Node.js is remarkably efficient at non-blocking network I/O, we cannot definitively claim it outperforms Native C++ holistically without also measuring the heavy browser UI CPU tax.

D. The Cost of String Concatenation and DOM Manipulation

The Naive Web implementation severely struggled with rendering latency, experiencing near-total client freezes under the full 5,000 msg/sec load (dropping to <10 FPS). This failure was entirely localized to the browser's frontend JavaScript execution.

Crucially, this variant exposes a classic frontend performance bug: the upstream naive template logs the *entire message history* by constantly reading, concatenating, and rewriting the full contents of an unbounded `<textarea>` element. At 5,000 messages per second, this unbounded string concatenation results in a quadratic workload scaling. The overhead of repeatedly rewriting this ever-growing string buffer drove the browser's main thread into a severe backlog.

E. Motivating Observation: Naive Redraw Behavior

Prior to formally implementing the `g_timeout_add` 60Hz lock for the $N = 10$ trials, we informally observed that a naive C++ implementation (attempting to redraw the UI for every incoming packet) resulted in event loop saturation and near 100% CPU utilization. By separating the network ingress from the UI polling, we observed CPU utilization fall sharply.

F. Extended Observation: Async Deadlocks & Unbounded Queue Growth

While the formal 60-second trials show Flask backend memory reaching approximately 175 MB, extending the same backend's runtime to 5 minutes under continuous 5,000 msg/sec

load revealed unbounded memory growth past 1 GB RES. Post-mortem analysis revealed the root cause was a catastrophic concurrency deadlock intrinsic to Python's fragmented async ecosystem.

When installed, `Flask-SocketIO` automatically defaults to the cooperative `eventlet` async worker. However, because standard library modules were left explicitly unpatched in our test environment, Paho MQTT's background listener executed within a standard blocking OS thread. As Paho rapidly handed off 5,000 messages per second to the `eventlet`-driven web queues, the unpatched blocking I/O starved the cooperative greenlets. Because the dispatcher was locked out, the underlying `python-engineio` queues ballooned infinitely, rapidly leaking memory until the process crashed.

IV. THREATS TO VALIDITY AND ALTERNATIVE WEB FRAMEWORKS

A potential threat to the validity of this study is the specific web architecture initially chosen (Flask-SocketIO to a DOM-heavy HTML client). To preempt this critique, we formally built and tested a highly optimized web dashboard utilizing HTML5 Canvas and `requestAnimationFrame`.

As shown in our results, moving to a non-blocking Node.js backend significantly reduced the CPU routing overhead (dropping from Python's 53.9% down to an impressive 24.7%). Furthermore, the Canvas architecture successfully restored the client framerate to the 60 FPS vsync cap.

However, the fundamental architectural critique remains: deploying a full browser engine (Chromium/Electron) onto a dedicated workstation introduces a permanent, inescapable tax of hundreds of megabytes of RAM. While employing embedded runtimes like WebView2 or CEF could theoretically reduce some desktop-Chrome overhead, they still carry the baseline multi-process Chromium architecture, ensuring a massive RAM delta compared to a single-binary native application.

Sensitivity Caveat: All results are reported at a single fixed workload (5,000 msg/sec, 60s); we did not characterize behavior across different message rates or payload sizes.

V. CONCLUSION

For dedicated factory monitoring workstations, developers frequently default to deploying Web-based dashboards wrapped in browser engines. This paper demonstrates a profound architectural trade-off in telemetry visualization under extreme Industry 4.0 workloads. When subjected to a true 5,000 msg/sec data flood, the Full-Stack Web architecture (Node.js + Canvas) demonstrated highly efficient backend routing.

Crucially, the native C++ architecture completely avoids the massive browser memory tax, saving nearly 300 MB of physical RAM compared to the Node.js web stack. Therefore, the choice of architecture is a strict hardware trade-off: while web architectures offer rapid development and decouple routing from rendering, they introduce a permanent, multi-hundred-megabyte memory tax. Deployments constrained by strict RAM limits should favor unified native architectures to reclaim that critical memory headroom.

ACKNOWLEDGMENTS

This work was supported by independent research and the open-source community. The author would like to thank the developers of the `Flask-MQTT` library and the broader GTK and Node.js ecosystems for their invaluable tools and documentation.

REFERENCES

- [1] K. R. Priya *et al.*, "Edge Computing Powered Embedded Vision System for Real-Time Industrial Automation and Fault Detection," in *2025 International Conference on Metaverse and Current Trends in Computing (ICMCTC)*, IEEE, 2025. DOI: 10.1109/ICMCTC62214.2025.11196733
- [2] G. Jocher, A. Chaurasia, and J. Qiu, *Ultralytics YOLOv8* (Version 8.0.0) [Computer software], 2023. Available: <https://github.com/ultralytics/ultralytics>
- [3] T. Gao *et al.*, "Optimization of Industrial Quality Inspection Systems in Computer Vision: Integration of Deep Learning and IoT Technologies," *Journal of Organizational and End User Computing*, vol. 37, IGI Global, 2025. DOI: 10.4018/JOEUC.395859

Divyam Agarwal is a second-year undergraduate student in the Department of Production and Industrial Engineering at Delhi Technological University. His current research interests include high-performance computing, systems architecture, and real-time industrial automation.